SESSION 04 / PRINTABLE GUIDE

PySpark field guide: Common data engineering functions

Session 04 | Practical reference, not an additional timed lesson
Purpose: Choose the right API for importing, exploring, cleaning, validating, joining, and publishing data.

How to use this guide

Look up the task first, then the API. Each table explains **where/when** to use an operation and **why** it helps. Short recipes connect the APIs to the Session 04 device example.

This is a working vocabulary, not an exhaustive Spark API catalog. Start with built-in expressions; add custom Python only when the requirement cannot reasonably be expressed with them.

Most examples use long-established APIs. `try_to_timestamp` requires Spark 3.5 or later; Delta and Fabric connector APIs have separate dependencies. Inspect the actual runtime rather than assuming an API exists because a newer documentation page lists it.

Recognize the different API families

Notation	What it is	Example
`F.name(...)`	A function from `pyspark.sql.functions`, usually creating a Spark Column expression	`F.upper(F.col("device_id"))`
`df.method(...)`	A DataFrame operation returning another DataFrame, a result, or metadata	`df.select("device_id")`
`F.col("x").method(...)`	A Column operation, not a Python scalar/string method	`F.col("temperature_c").isNull()`
`spark.read...` / `df.write...`	Reader/writer builders that configure I/O	`spark.read.schema(schema).csv(path)`
`T.StructType(...)`	A schema/type definition from `pyspark.sql.types`	Explicit nested JSON schema
`Window...` and `.over(...)`	Analytical window specifications applied to Column expressions	Count repeated event keys without collapsing rows
`DeltaTable...`	Delta Lake API, not core `pyspark.sql.functions`	Controlled merge into an existing Delta table

Notation	What it is	Example
`display(...)`	Fabric notebook visualization helper, not a PySpark function	Table/chart exploration
`.synapsesql(...)`	Fabric connector extension, not a standard Apache Spark reader/writer method	Verified Warehouse staging handoff

Read before running recipes

The short recipes below deliberately use the **micro-fixture** tables to keep individual APIs explainable. For the primary realistic workload and full-batch examples, use the main notebook lab. Its `s04_lab_` tables, eleven-column CSV, and counts are separate from this reference's setup.

Complete the device notebook first, using its current nine-column CSV fixture. This reference reads its isolated Bronze/Silver tables. Run this setup, then choose the recipe you need; this is not a single new production pipeline.

```python
from pyspark.sql import functions as F, types as T, Window

print("Spark runtime:", spark.version)
spark.conf.set("spark.sql.session.timeZone", "UTC")

raw = spark.table("s04_device_events_bronze").filter(
    F.col("batch_id") == "s04-devices-001"
)
events = spark.table("s04_device_events_silver").filter(
    F.col("batch_id") == "s04-devices-001"
)
```

`raw` has nine source events with raw strings. `events` has four accepted events with typed UTC timestamps and decimal temperatures. These are different contracts; do not casually interchange them.

A transformation normally returns a new DataFrame; it does not change the original variable or persist a table. Spark evaluates most transformations lazily. Actions such as `count()`, `show()`, `collect()`, and writes trigger work; an assigned variable alone is not a completed load.

Task index

1. Import data and define schemas
2. Explore and profile data
3. Select, name, and derive columns
4. Clean strings and identifiers

5. Handle types, nulls, and conditional rules
6. Filter and route records
7. Work with dates and timestamps
8. Parse JSON, arrays, and structures
9. Aggregate and calculate metrics
10. Join, combine, and compare datasets
11. Use windows, sequence, and deduplication policies
12. Validate quality and preserve lineage
13. Write data and manage table changes
14. Control execution, memory, and custom logic

1. Import data and define schemas

API	Where/when	Why use it / important boundary
`spark.read.csv(path)`	Delimited source exports	Read many files as a distributed DataFrame. Explicitly configure header, delimiter, quoting, and schema; CSV is not self-describing.
`spark.read.json(path)`	JSON documents or JSON Lines	Parse nested data directly when its format/schema is understood. Use `multiLine=True` for a whole multiline document, not automatically for JSON Lines.
`spark.read.text(path)`	Preserve one raw text record per line	Keep the exact payload string before parsing. Useful for the order JSON Lines example; not a whole-file parser by default.
`spark.read.parquet(path)`	Typed columnar files	Reuse embedded types and columnar reads. Still validate that the schema and source scope match the contract.
`spark.read.format("delta").load(path)`	Existing Delta table path	Read a Delta snapshot through its transaction log. Do not treat its underlying Parquet directory as equivalent.
`spark.table(name)`	Registered Lakehouse/Spark tables	Read the logical table in the current catalog/Lakehouse context. This does not read an arbitrary Warehouse table by name.

API	Where/when	Why use it / important boundary
Reader `.schema(...)`	Stable, governed input contracts	Avoid accidental inference and preserve invalid business strings until explicit conversion. A schema is not full business validation.
Reader `.option(...)` / `.options(...)`	Format-specific parsing behavior	Make assumptions visible: `header`, `sep`, `mode`, `multiLine`. Options are reader/format-specific, not universal flags.
`T.StructType`, `T.StructField`	Flat or nested schemas	Define field names, types, and structure explicitly instead of guessing from a small sample. Validate required values separately.
`T.StringType`, `IntegerType`, `DecimalType`, `TimestampType`, `ArrayType`	Source and target type contracts	Represent identifiers as strings, exact amounts as decimals, and nested arrays as arrays. Type choice changes meaning and supported operations.
`F.input_file_name()`	File-based ingestion	Attach a source path for lineage during the file read; it is not a stable business key or an ingestion timestamp.
`spark.createDataFrame(...)`	Small tests and reference lookups	Create controlled synthetic inputs or tiny mappings. Do not build a huge Python list on the driver as an ingestion strategy.

Recipe: Read the expanded CSV without guessing business types.

PYTHON

```python
source_fields = [
    "device_id", "event_id", "event_time_utc", "temperature_c",
    "firmware_version", "signal_strength_dbm",
    "ui_theme", "debug_message", "export_row_number",
]
landing_schema = T.StructType([
    T.StructField(name, T.StringType()) for name in source_fields
])

file_rows = (
    spark.read.schema(landing_schema)
    .option("header", True)
    .option("mode", "FAILFAST")
    .csv("Files/session_04/devices/batch-001/device_readings.csv")
    .withColumn("source_file", F.input_file_name())
    .withColumn("batch_id", F.lit("s04-devices-001"))
    .withColumn("ingested_at_utc", F.current_timestamp())
)
```

`FAILFAST` handles parser problems, not the domain validity of `sensor error`. Inspect header/order and the complete source scope. Do not scan a folder containing baseline, corrected, and expected-output files indiscriminately.

2. Explore and profile data

API	Where/when	Why use it / important boundary
`df.printSchema()`	First look at a source or output	See nested structure and actual data types before writing expressions. A successful schema print does not prove a successful full read.
`df.columns` , `df.dtypes` , `df.schema`	Column inventory and contract checks	These are properties, not functions. Use them to detect unexpected/ missing fields and inspect types.
`df.show(n, truncate=False)`	Small text previews	Inspect values and whitespace evidence without collecting the entire source. Preview rows are not a full-data profile.
`df.limit(n)`	Bound inspection or driver-side visualization	Cap rows before `toPandas()` / `collect()` . It is not representative random sampling and does not guarantee a cheap upstream query.

API	Where/when	Why use it / important boundary
`df.count()`	Exact row-count evidence	Count all rows in the selected DataFrame. It is an action and can be expensive; avoid repeatedly scanning just to print progress.
`df.describe(...)`	Quick basic statistics	Inspect numeric/string summaries. Raw text columns do not become valid numeric measures because they have statistics.
`df.summary(...)`	More detailed descriptive statistics	Request counts, mean, standard deviation, min/max, and quartiles. Percentiles/quantile methods can be approximate.
`df.sample(fraction=..., seed=...)`	Exploratory samples of large data	Inspect a controlled random fraction. A seed helps repeatability but is not a source snapshot; sample size is not an exact row guarantee.
`df.approxQuantile(...)`	Distribution and outlier investigation	Estimate quantiles with an explicit error tolerance. Domain rules, not quantiles alone, determine rejection.
`F.countDistinct(...)`, `F.approx_count_distinct(...)`	Key cardinality/profile	Distinguish exact distinct counts from estimates. Null handling matters; approximate counts are not uniqueness proofs.
`df.groupBy(...).count()`	Frequencies and repeated keys	Find skew, categories, and duplicate groups while preserving their counts. Often more informative than a preview.

Recipe: Profile the whole selected batch, not just displayed rows.

PYTHON

```python
raw.agg(
    F.count("*").alias("raw_rows"),
    F.countDistinct("device_id").alias("raw_device_spellings"),
    F.countDistinct(F.upper(F.trim("device_id"))).alias("normalized_devices"),
).show()

raw.groupBy("firmware_version").count().orderBy("firmware_version").show()
```

Expect 9 rows, 3 device spellings, and 2 normalized devices. For missing/invalid/range profiles and charts, use the exploration lesson.

3. Select, name, and derive columns

API	Where/when	Why use it / important boundary
`F.col(name)`	Reference a Spark column	Build expressions such as `F.col("quantity") * F.col("price")`. It is not the current row's Python value.
`F.lit(value)`	Add a constant or compare against a literal	Distinguish a string value from a column name; useful for batch labels and rule versions.
`df.select(...)`	Silver/transfer allowlists and expression projection	Choose exactly what crosses a layer boundary. Preferred over keeping every source/helper column by default.
Column `.alias(name)`	Name a derived output column	Make calculations and aggregates understandable to the next reader.
`df.withColumn(name, expression)`	Add or replace one column	Use for a small sequence of transformations; many repeated calls can create an unnecessarily large plan.
`df.withColumnRenamed(old, new)`	Standardize a source field name	Rename without changing its values/ type. Review joins and consumers affected by the new name.
`df.drop(...)`	Remove known temporary fields	Useful in a working DataFrame; a final allowlist better prevents unexpected new source fields leaking into Silver.
`df.selectExpr(...)` , `F.expr(...)`	Express Spark SQL calculations	Useful for familiar SQL syntax or `try_cast` . These are Spark SQL expressions, not Warehouse T-SQL or a parameterized SQL interface.

Recipe: A Silver projection is deliberate.

PYTHON

```python
silver_contract = events.select(
    "batch_id", "device_id", "event_id",
    "event_time_utc", "temperature_c",
    "firmware_version", "signal_strength_dbm",
    "source_file", "ingested_at_utc", "quality_rule_version",
)
```

Do not use `select("*")` as the default Gold contract. See the column decision guide.

4. Clean strings and identifiers

API	Where/when	Why use it / important boundary
`F.trim`, `F.ltrim`, `F.rtrim`	Unwanted edge spaces	Normalize boundary whitespace under an agreed policy. Trimming does not remove all embedded/ unicode whitespace problems.
`F.upper`, `F.lower`	Domain-approved case normalization	Align identifiers or categories. Do not change case-sensitive identifiers without approval.
`F.length`	Empty/length validation	Detect blank-after-trim values or fields exceeding a serving contract. Null and empty string are different.
`F.regexp_replace`	Known text patterns	Replace an agreed pattern, such as repeated spaces. Do not strip arbitrary characters and assume an invalid identifier became valid.
`F.regexp_extract`	Extract a known structured token	Parse a capture group from patterned text. A no-match can produce an empty string; test it explicitly.
Column `.rlike(pattern)`	Pattern validation	Require a documented identifier or numeric format before acceptance. Use explicit anchors for a whole-string contract.
`F.split`	Delimited content inside a field	Produce an array for later processing. The separator is a regex, not always a literal delimiter.
`F.substring`	Fixed-position source codes	Extract agreed segments; Spark substring positions are one-based.
`F.concat`, `F.concat_ws`	Labels or composed text	Combine fields. `concat_ws` skips null inputs, so it is not automatically a collision-safe business-key construction.
`F.translate`	Simple character-by-character mapping	Useful for an approved transliteration/removal rule; it is not a regex replacement or a general address-cleaning solution.

Recipe: Normalize without hiding the raw source.

```python
normalized = (
    raw.withColumn("device_id", F.upper(F.trim("device_id")))
    .withColumn("event_id", F.upper(F.trim("event_id")))
    .withColumn("temperature_text", F.trim("temperature_c"))
)
```

Bronze remains unchanged because no write occurred. A value becoming syntactically tidy does not establish that it belongs to a valid business entity.

5. Handle types, nulls, and conditional rules

API	Where/when	Why use it / important boundary
Column `.cast(type)`	Convert trusted/validated values	Establish real numeric/date types. Malformed input and overflow can raise errors under ANSI behavior; do not assume every cast returns null.
`F.expr("try_cast(...)")`	Tolerant parsing of untrusted strings	Use Spark SQL's supported `try_cast` to produce null on supported failed value conversions, then classify them. Unsupported type combinations can still fail.
Column `.isNull()`, `.isNotNull()`	Missingness and parsed-value checks	Test null with column expressions, not Python `is None` or equality to `None`.
`F.isnan(...)`	Floating-point NaN quality checks	NaN is different from null. Use only with appropriate numeric types.
`F.when(...).otherwise(...)`	Classification and conditional values	Encode an ordered business rule. The first matching condition wins; omitting `otherwise` leaves unmatched rows null.
`F.coalesce(...)`	First non-null value among columns	Apply an approved fallback. A fallback can hide a quality problem if chosen without a business rule.
`df.fillna(...)` / `df.na.fill(...)`	Approved missing-value defaults	Fill nulls of compatible types. Does not repair arbitrary invalid strings, and "unknown amount = 0" is not a safe universal rule.

API	Where/when	Why use it / important boundary
`df.dropna(...)` / `df.na.drop(...)`	Explicit exclusion policy for null rows	Remove only under an approved row-retention rule. Quarantine evidence may be required instead of deletion.
`F.round`, `F.bround`	Approved numeric rounding	`round` uses HALF_UP; `bround` uses HALF_EVEN. Specify the business policy and keep exact decimals for amounts.
`F.abs`, `F.greatest`, `F.least`	Differences, bounds, and comparisons	Calculate agreed derived values. Do not silently clamp an invalid measurement into the accepted range.

Recipe: Failed parsing is evidence, not zero. Run after the string recipe.

PYTHON

```python
typed = (
    normalized.withColumn(
        "temperature_value", F.expr("try_cast(temperature_text as decimal(8,2))")
    )
    .withColumn(
        "event_timestamp", F.expr("try_cast(event_time_utc as timestamp)")
    )
)

parse_diagnostic = typed.withColumn(
    "parse_issue",
    F.when(F.col("temperature_value").isNull(), F.lit("UNUSABLE_TEMPERATURE"))
    .when(F.col("event_timestamp").isNull(), F.lit("UNUSABLE_TIMESTAMP"))
    .otherwise(F.lit("PARSED")),
)
parse_diagnostic.select("event_id", "temperature_text", "parse_issue").show()
```

This diagnoses parsing only. `PARSED` does not mean accepted: duplicates, key/format rules, and range checks still apply. The actual device lesson owns the complete acceptance policy.

6. Filter and route records

API	Where/when	Why use it / important boundary
`df.filter(...)` / `df.where(...)`	Select a scope or route	Equivalent DataFrame filters; use before expensive work when valid for the business contract.

API	Where/when	Why use it / important boundary
Column `.isin(...)`	Approved category membership	Express a short allowed set. Use a lookup/join for a large governed reference list instead of a huge Python list.
Column `.between(low, high)`	Inclusive range checks	State a domain range clearly; null does not become false automatically in all compound logic.
Column `.contains`, `.startswith`, `.endswith`	Simple text predicates	Prefer readable literal predicates when regex is unnecessary.
`&`, `|`, `~` on Columns	Combine conditions	Use parentheses around comparisons; Python `and` / `or` / `not` are not Spark Column operators.
Column `.eqNullSafe(other)`	A comparison where two nulls should be equal	Useful for defined comparison semantics. It does not make missing identifiers valid join keys.

Recipe: Explicitly keep unusable and out-of-range readings for inspection. Run after `typed`.

```python
measurement_review = typed.filter(
    F.col("temperature_value").isNull()
    | ~F.col("temperature_value").between(-40, 85)
)
measurement_review.select("event_id", "temperature_text").show()
```

Expect E005 and E007. E006 has a timestamp issue and E002 has duplicate-key ambiguity; this measurement-only filter intentionally does not detect every problem.

Filters retain rows for which the condition is true, not null. A predicate such as `temperature < 85` does not retain rows with missing temperature for later review unless you include that branch.

7. Work with dates and timestamps

API	Where/when	Why use it / important boundary
`F.to_date(...)`	Business-date derivation	Derive a date from a valid timestamp, or parse trusted date text with a format. Invalid text behavior depends on runtime/ configuration.

API	Where/when	Why use it / important boundary
`F.to_timestamp(...)`	Parse validated timestamp text	Supply the source format where needed. Do not assume malformed input is always tolerated.
`F.try_to_timestamp(...)`	Tolerant timestamp parsing on Spark 3.5+	Returns null for failed parsing; a format expression can be passed with `F.lit(...)`. Verify actual runtime support.
`F.current_timestamp()`, `F.current_date()`	Run/publication metadata	Record processing time/date. These are not source event time and do not prove the batch's business period.
`F.date_trunc(...)`, `F.trunc(...)`	Calendar bucket boundaries	`date_trunc` returns timestamp buckets; `trunc` returns date truncation. Name the intended grain and timezone.
`F.year`, `F.month`, `F.dayofmonth`, `F.hour`	Calendar dimensions or grouping	Extract parts from typed dates/ timestamps. Do not discard the original timestamp needed for lineage and ordering.
`F.datediff(end, start)`, `F.date_add`, `F.date_sub`	Calendar-day intervals	Calculate day-based aging and boundaries. Day difference is not elapsed fractional hours.
`F.date_format(...)`	Display/export labels	Produce formatted strings such as `yyyy-MM`. Keep native date/ timestamp fields for joins, ordering, and arithmetic.
`F.to_utc_timestamp`, `F.from_utc_timestamp`	A known source/target timezone conversion	Convert under an explicit timezone contract. Do not shift an already correctly interpreted UTC instant a second time.
`F.window(timestamp, duration)`	Time-bucket grouping	Group events into time windows such as hourly buckets. This differs from analytical `Window.partitionBy`; a batch grouping is not a complete streaming design.

Recipe: Derive the daily grain from typed UTC Silver events.

```python
dated_events = events.select(
    "batch_id", "device_id", "event_id", "temperature_c",
    F.to_date("event_time_utc").alias("reading_date"),
    F.date_trunc("hour", "event_time_utc").alias("reading_hour_utc"),
)
```

For this fixture, all accepted readings belong to 2026-10-28 UTC. A local-business-day requirement needs a deliberately defined timezone conversion before grouping, including daylight-saving behavior.

8. Parse JSON, arrays, and structures

API	Where/when	Why use it / important boundary
`F.from_json(text, schema)`	Parse a JSON string column	Create typed structs/arrays while retaining the raw payload. Parse success and schema compatibility are not complete business acceptance.
`F.to_json(...)`	Controlled serialization	Convert a struct/array/map to JSON for an explicit downstream contract. Do not serialize a whole internal row and accidentally expose helper/ sensitive fields.
`F.get_json_object(...)`	Extract one JSON path as text	Useful for lightweight inspection. Repeated extraction is less clear than one explicit schema when many fields are needed.
`F.struct(...)`	Build or retain a named nested record	Package related fields before serialization or array transformations.
`F.array(...)`, `F.size(...)`	Build/check array content	Inspect repeated items before expansion. Null/empty handling needs an explicit rule and can depend on configuration.
`F.explode(...)`	One output row per array/map element	Change grain intentionally. Empty/ null collections can remove the parent from the result.
`F.explode_outer(...)`	Expand while retaining empty/null evidence	Preserve a parent evidence row with null expanded values; that marker is not a real business item.

API	Where/when	Why use it / important boundary
`F.posexplode_outer(...)`	Expansion with element position	Preserve array position for investigation. Position is not automatically a stable business line key.
`F.transform(...)`, `F.filter(...)`	Higher-order array operations	Apply Column/lambda logic inside arrays without Python row UDFs. `F.filter` filters array elements; `df.filter` filters rows.
Column `.getField(...)`, `.getItem(...)`	Access nested fields or indexed content	Navigate a known structure. Array indexing and SQL-style element-access functions can use different conventions; verify the API.

Recipe: Keep an empty order visible, as in the main order walkthrough.

PYTHON

```python
json_schema = T.StructType([
    T.StructField("order_id", T.StringType()),
    T.StructField("lines", T.ArrayType(T.StructType([
        T.StructField("sku", T.StringType()),
        T.StructField("quantity", T.StringType()),
    ]))),
])
payloads = spark.createDataFrame(
    [
        ('{"order_id":"O1","lines":[{"sku":"PUMP-01","quantity":"2"}]}',),
        ('{"order_id":"O2","lines":[]}',),
    ],
    "raw_json string",
)
expanded = (
    payloads.withColumn("payload", F.from_json("raw_json", json_schema))
    .select("payload.order_id", F.explode_outer("payload.lines").alias("line"))
)
expanded.show(truncate=False)
```

Expect two evidence rows: O1's line and O2's null line marker. Do not count both as two valid order lines. Expanding two independent arrays can multiply rows; decide the intended relationship/grain before doing it.

9. Aggregate and calculate metrics

API	Where/when	Why use it / important boundary
`df.groupBy(...).agg(...)`	Silver-to-Gold grain change	Define one row per group and name every metric. Grouping by an unnecessary field silently changes the output grain.
`F.count("*")`	Row/event counts	Count all rows in the selected group. `F.count("column")` instead counts its non-null values.
`F.countDistinct(...)`	Business-key/entity counts	Count distinct non-null values/ combinations; do not confuse source rows, accepted events, and entities.
`F.sum` , `F.avg` , `F.min` , `F.max`	Approved numeric summaries	Compute metrics over a defined population. Most aggregates ignore null inputs; an empty/all-null aggregate is not automatically zero.
`F.stddev` , `F.variance`	Variability investigation	Understand spread; these use sample statistics. Use population variants when that is the approved definition.
`F.percentile_approx(...)`	Approximate median/percentile summaries	Summarize large distributions with declared accuracy. It is not an exact threshold certification.
`F.collect_list` , `F.collect_set`	Small per-group collections	Retain repeated/distinct values for a justified use case. Order is not guaranteed after shuffle and large groups can exhaust memory.
`GroupedData.pivot(...)`	A controlled wide summary	Turn known categories into columns. Prefer an explicit category list; uncontrolled cardinality can create a huge schema.

Recipe: The Gold preparation used in Session 04.

```python
daily = (
    events.groupBy(
        "batch_id",
        F.to_date("event_time_utc").alias("reading_date"),
        "device_id",
    )
    .agg(
        F.count("*").alias("reading_count"),
        F.min("temperature_c").alias("min_temperature_c"),
        F.max("temperature_c").alias("max_temperature_c"),
        F.avg("temperature_c").cast("decimal(8,2)").alias("avg_temperature_c"),
    )
)
daily.orderBy("device_id").show()
```

Expect two daily rows representing four accepted events. The means are 22.00 and 31.00. Do not average rounded group averages to invent a wider-grain measure; retain the required underlying sum/count when that is a consumer requirement.

10. Join, combine, and compare datasets

API	Where/when	Why use it / important boundary
`df.join(other, keys, how="left")`	Enrich facts with lookup/dimension attributes	Retain the left population while adding matches. Validate reference-key uniqueness to avoid multiplying facts.
`join(..., how="inner")`	Only matched records are required	Intentionally discard unmatched keys. Inappropriate when source-only exceptions must remain visible.
`join(..., how="full")`	Reconciliation or coverage comparison	Preserve keys from both sides; assess source/key quality before a raw-detail join.
`join(..., how="left_anti")`	Orphan/unmatched key checks	Return left rows with no match. Useful for rejected references and missing dimension membership.
`join(..., how="left_semi")`	Membership without enrichment	Retain left rows with a match without bringing right-side columns into the schema.
`df.alias(...)`, qualified `F.col(...)`	Joins with repeated column names	Make the intended source explicit and avoid ambiguous references.

API	Where/when	Why use it / important boundary
`df.unionByName(...)`	Combine compatible batches/ sources	Match fields by name. `allowMissingColumns=True` inserts nulls and needs explicit schema-evolution approval.
`df.union(...)`	Strict, position-aligned row combination	Retains duplicate rows and aligns by position, not name. Do not assume SQL UNION DISTINCT semantics.
`df.exceptAll(...)`	Compare complete business row multisets	Preserve duplicate differences. Align column order and types first; it resolves columns by position.
`df.intersect(...)` / `df.intersectAll(...)`	Common-row analysis	Choose distinct or duplicate-preserving behavior deliberately; neither is an arbitrary-key enrichment join.

Recipe: Enrich without changing the accepted event count.

PYTHON

```python
devices = spark.createDataFrame(
    [("D01", "North"), ("D02", "South")],
    "device_id string, site_name string",
)

duplicate_dimension_keys = devices.groupBy("device_id").count().filter(
    F.col("count") > 1
)
if duplicate_dimension_keys.limit(1).count():
    raise ValueError("Device reference keys are not unique.")

enriched = events.join(devices, "device_id", "left")
unmapped = events.join(devices, "device_id", "left_anti")
enriched.select("device_id", "event_id", "site_name").show()
```

Expect four enriched events and no unmapped events. Before trusting a real lookup, separately define blank/null-key policy, unknown members, and any effective-date/version rules.

11. Use windows, sequence, and deduplication policies

API	Where/when	Why use it / important boundary
`Window.partitionBy(...)`	Per-key/per-entity analytical calculation	Group rows for an expression without collapsing them. It is not physical output `partitionBy` or Spark execution `repartition`.
Window `.orderBy(...)`	Sequence within each analytical group	Define ordering for lag, rank, or running metrics. A timestamp tie needs an approved tie-breaker when choosing one row.
Column `.over(window)`	Apply an aggregate/window expression	Attach per-key evidence such as duplicate count to every source row.
`F.row_number()`	Ordered row numbering or approved winner selection	Returns one sequence number per row. Selecting 1 is only correct after a deterministic, business-approved ordering policy.
`F.rank()`, `F.dense_rank()`	Ranking with ties	`rank` leaves gaps after ties; `dense_rank` does not. Choose the business meaning, not whichever removes duplicates.
`F.lag(...)`, `F.lead(...)`	Compare neighboring observations	Retrieve previous/next values under the defined ordering. First/last boundary rows need a null/default interpretation.
Window `.rowsBetween(...)`	Running or row-count-based calculations	Frame by row position, not elapsed clock time. Specify the frame rather than relying on an unintended default.
Window `.rangeBetween(...)`	Value-range-based windows	Define a range over an ordering value under Spark's type/order restrictions. A count of rows is not a duration.
`df.distinct()`	Remove exact duplicate rows	Use when full-row set semantics are the intended contract, not when duplicate keys have differing attributes.
`df.dropDuplicates(subset)`	Keep one occurrence per subset	Does not define which conflicting non-key row wins. Unsafe as a substitute for quarantine or an approved latest-record policy.

Recipe: Preserve duplicate evidence before deciding what to publish.

```python
key_window = Window.partitionBy(
    F.upper(F.trim("device_id")),
    F.upper(F.trim("event_id")),
)
with_key_counts = raw.withColumn(
    "event_key_count", F.count("*").over(key_window)
)
ambiguous = with_key_counts.filter(F.col("event_key_count") > 1)
ambiguous.select("device_id", "event_id", "event_key_count").show()
```

Expect both E002 rows. This works over the already scoped one-batch `raw`; a multi-batch contract must specify whether batch is part of identity. Session 04 quarantines all ambiguous occurrences rather than choosing an arbitrary winner.

Recipe: Calculate ordered changes on already accepted events.

```python
sequence = Window.partitionBy("batch_id", "device_id").orderBy(
    F.col("event_time_utc").asc(), F.col("event_id").asc()
)
changes = (
    events.withColumn("previous_temperature", F.lag("temperature_c").over(sequence))
    .withColumn(
        "temperature_change",
        F.col("temperature_c") - F.col("previous_temperature"),
    )
)
changes.select(
    "device_id", "event_id", "temperature_c",
    "previous_temperature", "temperature_change",
).orderBy("device_id", "event_id").show()
```

The first accepted observation per device has no previous value. The next accepted changes are +4.00 for D01 and +2.00 for D02. These are changes between accepted observations, not proof of regular sampling or a change per fixed time interval.

12. Validate quality and preserve lineage

API / pattern	Where/when	Why use it / important boundary
`groupBy(keys).count().filter(count > 1)`	Uniqueness gate	Expose duplicate-key groups instead of assuming a declared schema enforces uniqueness.

API / pattern	Where/when	Why use it / important boundary
`F.sum(F.when(condition, 1).otherwise(0))`	Count rule failures	Compute explicit invalid/missing conditions in Spark; name the condition and population.
`df.isEmpty()`	Existence checks on supported Spark versions	Ask whether any rows exist without needing a full exact count; execution can still require a job.
`df.limit(1).count()`	Compatible existence gate	Return at most one row count when you only need yes/no. It is not a full profile.
`F.sha2(...)`	Approved content fingerprint	Useful for a versioned canonical-content comparison. Define field order, types, nulls, and serialization; a hash is not source readiness or anonymization.
`F.lit(...)` , `F.current_timestamp()`	Batch/rule/run metadata	Attach controlled labels and processing times. A label does not validate the associated input pair or enforce concurrency.
`df.createOrReplaceTempView(...)` , `spark.sql(...)`	SQL-oriented exploration in the Spark session	Reuse Spark DataFrames with Spark SQL. A session temp view is not a Warehouse view or a persisted dataset.
`df.exceptAll(...)` in both directions	Rerun/expected-result comparisons	Detect missing, unexpected, or duplicate business rows, excluding only metadata deliberately allowed to change.

Recipe: Compare the accepted business keys with an expected fixture.

PYTHON

```python
expected_keys = spark.createDataFrame(
    [("D01", "E001"), ("D01", "E003"), ("D02", "E004"), ("D02", "E008")],
    "device_id string, event_id string",
)
actual_keys = events.select("device_id", "event_id")

if (
    actual_keys.exceptAll(expected_keys).limit(1).count()
    or expected_keys.exceptAll(actual_keys).limit(1).count()
):
    raise ValueError("Accepted business keys differ from the expected fixture.")
```

This checks keys and duplicate multiplicity, not every temperature or metadata value. A complete expected-result check includes the required business columns in a fixed order.

Recipe: Use familiar SQL without changing engines.

```python
events.createOrReplaceTempView("s04_accepted_events_view")
spark.sql("""
    SELECT device_id, COUNT(*) AS accepted_event_count
    FROM s04_accepted_events_view
    GROUP BY device_id
    ORDER BY device_id
""").show()
```

This is Spark SQL. It does not open a Warehouse transaction, write through the Lakehouse SQL analytics endpoint, or bypass the Warehouse handoff.

13. Write data and manage table changes

API	Where/when	Why use it / important boundary
`df.write.format("delta")`	Persist Lakehouse Delta output	Select the Delta provider. One committed table write is not a transaction across Bronze, Silver, quarantine, and Warehouse Gold.
Writer `.mode("errorifexists")`	A new controlled target	Fail rather than silently overwrite an existing dataset. A useful safety default for a demonstration.
Writer `.mode("append")`	Truly additive records with replay controls	Add rows; it does not deduplicate, update, or make retries idempotent.
Writer `.mode("overwrite")`	Approved complete-snapshot replacement	Replace only a known isolated/owned target. Scope, schema, recovery, and concurrent-reader/writer behavior need review.
Writer `.saveAsTable(name)`	Registered Lakehouse table	Persist through the current catalog/Lakehouse. It does not create Warehouse Gold.
Writer `.save(path)`, `.parquet(path)`, `.csv(path)`	Explicit file/table-path output	Write distributed output, usually multiple part files. It is not generally one human-named CSV file.

API	Where/when	Why use it / important boundary
Writer `.partitionBy(...)`	Physical directory partitioning	Use a justified, bounded-cardinality column such as a date where workload/volume warrants it. Avoid tiny partitions by event ID.
Writer `.option("overwriteSchema", "true")`	Reviewed Delta contract replacement	Permit an intentional schema replacement in a controlled overwrite. Not a general fix for unexpected drift or permission problems.
`DeltaTable.forName(...)`, `.merge(...)`	Delta upserts under an approved policy	Delta-specific API for match/update/insert behavior. Require unique/valid source keys, defined updates/deletes, and safe retries; not a Warehouse T-SQL MERGE.
Fabric `.write.synapsesql(...)`	Verified Spark-to-Warehouse transfer	Fabric-specific extension with runtime/authentication constraints. Use staging and subsequent validation, not a blind overwrite of live Gold.

Opt-in write recipe: Only a new isolated demonstration table.

```python
(
    events.select(
        "batch_id", "device_id", "event_id", "event_time_utc", "temperature_c"
    )
    .write.format("delta")
    .mode("errorifexists")
    .saveAsTable("s04_reference_events_demo")
)
```

Do not run this against a shared target or change the mode merely to make a rerun green. It intentionally fails if the demo table already exists. The actual Session 04 publication path remains Silver -> transfer -> Warehouse staging -> validated Gold.

14. Control execution, memory, and custom logic

API	Where/when	Why use it / important boundary
`df.explain(mode="formatted")`	Investigate an expensive plan	Inspect scans, joins, exchanges, and execution strategy before adding tuning guesses.

API	Where/when	Why use it / important boundary
`df.cache()` , `df.persist(...)`	Reuse an expensive intermediate several times	Avoid recomputation when measurements justify storage cost. These are lazy until materialization; they are not durable source snapshots.
`df.unpersist()`	After reuse is finished	Release cached resources rather than holding unnecessary data for the session.
`df.repartition(...)`	Change distribution or increase parallelism	Causes a shuffle; use based on data size, skew, and downstream requirements, not a universal fixed partition count.
`df.coalesce(n)`	Usually reduce partition count	Typically avoids a full shuffle when reducing partitions but can reduce parallelism too far. Different from `F.coalesce` for null values.
`F.broadcast(small_df)`	A genuinely small lookup side of a join	Hint broadcast when it fits resources and the plan warrants it. Broadcasting a large table can create memory failures.
`df.orderBy(...)` / `df.sort(...)`	Required global ordering or bounded display	Sort deliberately; global sorting is expensive and does not make stored table row order a serving guarantee.
`df.sortWithinPartitions(...)`	A justified local ordering requirement	Orders each partition, not the whole DataFrame. It does not replace a global order for sequential logic.
`df.collect()` , `df.toPandas()`	Small, bounded results	Move data to driver memory. Aggregate/filter/limit first; never collect a large raw source just to loop over rows.
`df.transform(function)`	Reusable DataFrame transformation recipes	Compose functions accepting/ returning DataFrames while keeping Spark expressions. Different from array-level `F.transform` .
`F.udf(...)` , `F.pandas_udf(...)`	Necessary custom logic unavailable as suitable built-ins	Specify types and test behavior/ dependencies. Python/Arrow boundaries and serialization add cost; pandas UDF batches do not justify arbitrary side effects.

Recipe: Bound the driver-side data used for exploration.

```python
small_plot_input = (
    events.select("device_id", "event_time_utc", "temperature_c")
    .orderBy("device_id", "event_time_utc")
    .limit(1000)
    .toPandas()
)
```

This fixture has four accepted rows. For larger sources, a cap is not a representative sample and a preceding global sort can still be expensive. Filter to a meaningful scope or aggregate in Spark first.

Prefer built-in operations such as `F.upper` to a Python UDF implementing the same case conversion. Do not make external API calls or send records from a row UDF; retries, worker parallelism, data approval, and side effects require a separate controlled ingestion design.

A practical starting set

Task	Learn these first
Import	`spark.read`, `.schema`, `.option`, `spark.table`, `StructType`
Explore	`printSchema`, `show`, `count`, `groupBy().count`, `summary`
Clean	`select`, `col`, `lit`, `trim`, `upper`, `expr("try_cast(...)")`
Validate/route	`when`, `isNull`, `filter`, `between`, key counts with windows
Combine	`join`, `left_anti`, `unionByName`
Shape Gold	`groupBy().agg`, `count`, `sum`, `avg`, `to_date`
Publish/operate	`write.format("delta")`, `saveAsTable`, `exceptAll`, `explain`

Use the column guide to decide what to persist, not merely which functions are available. Knowing how to remove a column or duplicate is different from knowing whether the business contract permits it.

Common traps to recognize

- `F.count("*")` counts rows; `F.count("temperature_c")` counts non-null values; `countDistinct` counts a different population.
- `F.coalesce` handles null values; `df.coalesce` changes execution partitions.

- `F.filter` operates on array elements; `df.filter` selects rows.
- `Window.partitionBy` defines analytical groups; writer `partitionBy` defines storage layout; `repartition` changes execution distribution.
- A neat string, successful cast, or non-null field is not full business validation.
- `dropDuplicates` is not an approved conflict-resolution policy.
- A preview/sample, approximate metric, cache, or batch label is not full-data acceptance evidence.
- `saveAsTable` in a notebook does not mean Warehouse Gold is published.

Documentation and runtime boundaries

The Apache links below use Spark 3.5.6 as a versioned reference for the core vocabulary, not as a claim about the deployed Fabric runtime. Match documentation to `spark.version` and the actual enabled APIs.

- PySpark SQL functions
- DataFrame API
- DataFrameReader
- Window API
- Spark SQL null semantics
- Spark SQL ANSI behavior
- Delta merge documentation
- Fabric notebook visualization
- Fabric Spark connector for Warehouse

Core API documentation does not establish GCC High availability of Fabric UI or connectors. Validate the runtime, item access, identity, network path, and complete example in the target environment.

Source: 08_pyspark_function_reference.md. Companion-material links open the repository originals.